ENDPOINT-LEVEL BEHAVIORAL MONITORING AND RESPONSE FOR INDUSTRIAL CONTROL AND OT SYSTEMS

a project report submitted by

Safna S (URK22AI1104)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY
in
ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

under the supervision of

Mr. Rahul R, M.TECH., (Ph.D.)

DIVISION OF DATA SCIENCE AND CYBER SECURITY

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

KARUNYA INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed-to-be-University under Sec-3 of the UGC Act, 1956)
Karunya Nagar, Coimbatore - 641 114, India.

FEBRUARY 2026

Abstract

Industrial Control Systems and Operational Technology (ICS/OT) environments form the backbone of critical infrastructure sectors, including water treatment, power generation, and industrial manufacturing. The convergence of IT and OT networks has significantly increased the exposure of these systems to advanced cyber threats. Conventional endpoint detection and response (EDR) mechanisms, largely designed for enterprise IT environments, are not well suited for ICS/OT systems due to their deterministic operational behavior, legacy components, and strict real-time and safety constraints.

This work presents a lightweight, behavior-based Endpoint Detection and Response (EDR) framework tailored for ICS/OT environments. The proposed framework emphasizes endpoint-level behavioral monitoring by analyzing process execution characteristics, control command patterns, and time-based operational profiles, rather than relying on signature-based or resource-intensive detection techniques. A layered architecture is introduced comprising an ICS/OT endpoint layer, local behavioral data collection, baseline behavior modeling, and anomaly detection.

The effectiveness of the proposed approach is evaluated using the Secure Water Treatment (SWaT) dataset, which represents a realistic industrial control environment with both normal and attack scenarios. Behavioral baselines are constructed using normal operational data, and deviations observed during attack conditions are identified through rule-based and threshold-driven detection mechanisms. Experimental results indicate that the proposed framework can accurately identify anomalous behaviors while maintaining low computational overhead, demonstrating its suitability for practical deployment in ICS/OT environments.

Table of Contents

Chap. No	Title	Page No
	Abstract	
	Table of Contents	
1	Introduction	1
1.1	Objective	1
1.2	Problem Statement	2
1.3	Chapter-wise Summary	2
2	Literature Survey	3
2.1	Existing System	3
2.2	Inference from Existing Systems	5
2.3	System Performance	6
2.4	Limitations and Challenges	6
3	Proposed System	7
3.1	Detailed Design	7
3.2	Design of Methodology	8
3.3	Modules	9
3.4	Database Design	10
3.4.1	Entity Relationship Diagram	10
4	Implementation	13
4.1	Layer 1:	13

4.2 Layer 2 Implementation 14

4.3 Layer 3 Implementation 14

4.4 Layer 4 Implementation 14

5 Conclusion and Summary 15

References

Appendix 1

Appendix 2

Chapter -1
Introduction

Industrial Control Systems (ICS) and Operational Technology (OT) are integral to the operation of critical infrastructure such as water treatment facilities, power generation plants, manufacturing units, and transportation systems. These systems are responsible for monitoring and controlling physical processes and are designed to operate continuously with high reliability and strict timing constraints. Traditionally, ICS/OT environments were isolated from external networks; however, recent advancements in Industrial Internet of Things (IIoT), remote monitoring, and IT–OT convergence have significantly increased their connectivity and exposure to cyber threats.

The growing number of cyber attacks targeting industrial environments has highlighted the limitations of conventional security mechanisms. Many existing cybersecurity solutions, including antivirus and traditional endpoint detection and response (EDR) systems, are primarily designed for information technology (IT) environments. Such solutions often rely on signature-based detection techniques, frequent updates, and resource-intensive analysis, which are unsuitable for ICS/OT systems due to their deterministic behavior, legacy components, and safety-critical operations. Moreover, aggressive response actions such as system shutdown or forced isolation can disrupt industrial processes and lead to unsafe operating conditions.

To address these challenges, behavior-based security approaches have gained attention in recent years. ICS/OT systems typically exhibit predictable and repetitive operational patterns during normal operation. Any deviation from these patterns may indicate malicious activity, system misuse, or abnormal behavior. Leveraging this characteristic, this project focuses on the design of a lightweight, behavior-based Endpoint Detection and Response (EDR) framework tailored specifically for ICS/OT environments. The proposed system emphasizes endpoint-level behavioral monitoring, baseline modeling, and anomaly detection while ensuring minimal impact on operational safety. The Secure Water Treatment (SWaT) dataset is used to emulate real-world industrial processes and evaluate the effectiveness of the proposed approach.

1.1 Objective

The primary objective of this project is to design and develop a lightweight, behavior-based Endpoint Detection and Response (EDR) framework tailored for Industrial Control Systems and Operational Technology (ICS/OT) environments. The proposed framework aims to enhance the security of industrial endpoints by monitoring and analyzing operational behavior rather than relying on traditional signature-based detection mechanisms.

The specific objectives of this project are as follows:

- To study the security challenges and constraints associated with ICS/OT environments, including deterministic behavior, legacy systems, and safety-critical operations.
- To design a layered EDR architecture suitable for ICS/OT systems that focuses on endpoint-level behavioral monitoring and analysis.
- To extract and model normal operational behavior using industrial process data from the Secure Water Treatment (SWaT) dataset.

- To develop behavioral baseline profiles based on process execution patterns, control command frequency, and time-based operational characteristics.
- To implement anomaly detection mechanisms that identify deviations from established behavioral baselines using rule-based and threshold-driven techniques.
- To evaluate the effectiveness of the proposed framework in detecting anomalous behavior while maintaining low computational overhead and preserving operational safety.

These objectives collectively aim to demonstrate that behavior-based endpoint monitoring can provide an effective and practical security solution for ICS/OT environments without disrupting normal industrial operations.

1.2 Problem statement

Industrial Control Systems (ICS) and Operational Technology (OT) environments are increasingly exposed to cyber threats due to the growing integration of industrial networks with enterprise IT systems and remote monitoring technologies. Despite this increased exposure, many ICS/OT environments continue to rely on legacy security mechanisms that are insufficient to detect modern, sophisticated attacks. Conventional endpoint security and EDR solutions, which are primarily designed for IT systems, are not well suited for industrial environments due to differences in operational behavior, performance constraints, and safety requirements.

The key problems addressed in this project are summarized as follows:

- Existing EDR and endpoint security solutions are designed for IT environments and do not account for the deterministic and repetitive operational behavior of ICS/OT systems.
- Signature-based detection techniques are ineffective against novel or zero-day attacks commonly observed in industrial cyber incidents.
- Resource-intensive security mechanisms can introduce performance overhead, which is unacceptable in real-time and safety-critical industrial operations.
- Aggressive response actions, such as automatic shutdown or endpoint isolation, may disrupt industrial processes and lead to unsafe operating conditions.
- Limited visibility into endpoint-level operational behavior makes it difficult to detect control logic manipulation, command injection, and abnormal process execution.
- There is a lack of lightweight, behavior-based EDR frameworks specifically tailored for ICS/OT environments and validated using realistic industrial datasets.

These challenges highlight the need for a security solution that can effectively detect anomalous behavior at the endpoint level while preserving the safety, reliability, and performance requirements of ICS/OT systems.

1.3 Chapter wise summary

This report is organized into five chapters, each addressing a specific aspect of the proposed behavior-based Endpoint Detection and Response (EDR) framework for ICS/OT environments.

Chapter 1 introduces the background and motivation for securing Industrial Control Systems and Operational Technology environments. It presents the objective of the project, defines the problem statement, and highlights the need for a lightweight, behavior-based EDR solution tailored for industrial systems.

Chapter 2 provides a comprehensive literature survey of existing security mechanisms for ICS/OT environments. It reviews network-based, machine learning–based, rule-based, and system-level security approaches. The chapter further analyzes the performance, limitations, and challenges of existing systems, identifying research gaps that motivate the proposed solution.

Chapter 3 describes the proposed system in detail. It explains the overall system architecture, design methodology, functional modules, and database design. The chapter also presents the Entity Relationship diagram, illustrating the logical relationships among system components and stored data.

Chapter 4 focuses on the implementation of the proposed system. It explains the layer-wise implementation of the framework up to the anomaly detection stage, including endpoint emulation, data collection, behavior baseline modeling, and anomaly detection using mathematical formulations.

Chapter 5 concludes the report by summarizing the key contributions of the project and discussing the effectiveness of the proposed approach. It also outlines potential future enhancements and extensions to further improve ICS/OT endpoint security.

Chapter - 2
Literature survey

2.1 Existing System

Existing security solutions for Industrial Control Systems (ICS) and Operational Technology (OT) environments can be broadly categorized based on their focus area and detection methodology. The major categories identified from the literature are network-based anomaly detection, machine learning and deep learning–based approaches, behavioral and rule-based detection, and endpoint and system-level security mechanisms.

2.1.1 Network-Based Anomaly Detection Systems

Network-based anomaly detection systems focus on analyzing industrial network traffic to identify abnormal communication patterns.

Zare et al. (2024) proposed a real-time network-based anomaly detection system using a sequence-to-sequence LSTM autoencoder to model normal ICS traffic behavior. The model was evaluated using the SWaT dataset and demonstrated improved detection performance compared to traditional network anomaly detection systems. However, the approach relies heavily on deep learning techniques, which require high computational resources and large volumes of training data, limiting its suitability for real-time ICS environments.

Amer et al. (2025) introduced a Modbus-NFA behavior model that represents normal Modbus communication using a non-deterministic finite automaton. This model effectively detects anomalies by identifying deviations in traffic sequences. While the approach improves protocol-level behavior modeling, it is restricted to Modbus/TCP traffic and does not capture endpoint-level operational behavior.

2.1.2 Machine Learning and Deep Learning–Based Detection Systems

Several studies leverage machine learning and deep learning techniques to detect anomalies and cyber attacks in ICS environments.

Kuchar and Fujdiak (2025) proposed a data-driven anomaly classification framework using deep neural networks to classify industrial traffic into security, operational, and service anomalies. The approach integrates multiple data sources, including network traffic, sensor data, and hardware status. Despite improved detection granularity, the model incurs higher computational overhead and lacks explainability, which is critical in safety-sensitive industrial systems.

Imran et al. (2023) presented a performance analysis of machine learning-based defense strategies for detecting advanced persistent threats in ICS environments. The study demonstrated high detection accuracy using supervised learning models. However, the approach depends on labeled datasets and is prone to class imbalance issues, reducing its effectiveness in real-world deployment scenarios.

2.1.3 Rule-Based and Behavioral Modeling Approaches

Rule-based and behavioral approaches aim to model normal operational characteristics of ICS processes.

Zhu et al. (2025) proposed an invariant rule-based anomaly detection technique that learns relationships between sensors and actuators using association rule mining. Evaluated on the SWaT dataset, the approach effectively detects stealthy attacks by identifying violations of learned invariants. Nevertheless, the method requires careful parameter tuning and exhibits increased computational cost when applied to long-duration industrial time-series data.

2.1.4 Endpoint and System-Level Security Mechanisms

Endpoint-focused security mechanisms remain relatively limited in ICS research.

Chen et al. (2023) conducted a comprehensive survey on data management strategies for endpoint-based advanced persistent threat detection. The study highlighted challenges related to large-scale log generation, storage overhead, and fragmented data collection mechanisms in existing EDR systems. While the work provides valuable insights, it primarily focuses on IT environments and does not address ICS-specific operational constraints.

2.2 Inference from Existing Systems

From the detailed literature survey, it is evident that significant research efforts have been made to enhance the cybersecurity of Industrial Control Systems (ICS) through anomaly detection, intrusion detection, and traffic monitoring techniques. Most existing systems primarily focus on network-level monitoring, protocol-specific analysis, or machine learning–based detection mechanisms to identify malicious activities in industrial environments.

Network-based approaches, such as deep learning–driven anomaly detection and protocol behavior modeling, have shown promising results in identifying deviations in industrial traffic patterns. However, these approaches often require high computational resources, extensive training data, and protocol-specific knowledge, which limits their scalability and real-time applicability in resource-constrained ICS/OT environments. Similarly, SDN-based and traffic control mechanisms are effective in mitigating network-centric attacks like DDoS but introduce architectural complexity and do not provide visibility into endpoint-level operational behavior.

Machine learning and deep learning–based systems demonstrate high detection accuracy in controlled environments; however, they rely heavily on labeled datasets, suffer from explainability issues, and may generate false positives when deployed in dynamic industrial settings. Rule-based and invariant modeling approaches improve interpretability and can detect stealthy attacks but require careful parameter tuning and may incur computational overhead when applied to large-scale time-series data.

Overall, the inference from existing systems indicates a clear research gap in lightweight, behavior-based endpoint detection mechanisms specifically designed for ICS/OT environments. There is a strong need for solutions that can monitor endpoint-level operational behavior, maintain low overhead, and preserve industrial safety without disrupting normal processes.

2.3 System Performance

The performance of existing ICS security systems is generally evaluated using metrics such as detection accuracy, recall, false positive rate, and computational overhead. Many deep learning–based network anomaly detection systems report high detection accuracy and improved recall when evaluated on benchmark datasets such as SWaT. However, these performance gains often come at the cost of increased training complexity, longer detection latency, and higher resource consumption.

SDN-based defense mechanisms demonstrate improved system availability and reduced packet loss during network flooding attacks, but their performance is highly dependent on accurate traffic modeling and synchronized control infrastructure. Behavioral and rule-based systems provide improved interpretability and are effective in detecting logic-based and stealthy attacks, though their performance can degrade if the learned rules do not generalize well across different industrial setups.

Endpoint-focused and system-level approaches remain limited in performance evaluation, particularly in ICS contexts, due to challenges in collecting comprehensive endpoint logs without affecting real-time operations. As a result, existing systems often prioritize detection performance over operational safety. This highlights the need for a balanced approach that achieves reliable anomaly detection while maintaining low computational overhead and ensuring uninterrupted industrial operation.

2.4 Limitations and Challenges

Despite notable advancements, existing ICS security solutions face several limitations and challenges that restrict their practical deployment:

- Most approaches focus on network-level visibility, offering limited insight into endpoint-level operational behavior.
- Deep learning and machine learning–based systems require large volumes of training data, labeled datasets, and high computational resources.
- Lack of explainability in AI-based models reduces trust and usability in safety-critical industrial environments.
- Protocol-specific solutions lack generalization across heterogeneous ICS/OT setups.
- Aggressive detection and response mechanisms may disrupt industrial processes and compromise safety.
- High false positive rates can lead to alarm fatigue and reduced confidence in security systems.
- Limited consideration of deterministic and time-sensitive behavior inherent to ICS operations.

These challenges emphasize the necessity for a lightweight, behavior-based EDR framework that focuses on endpoint-level operational characteristics, minimizes overhead, and ensures OT-safe operation. Addressing these limitations forms the foundation of the proposed system presented in the subsequent chapters.

Chapter - 3
Propose System

3.1 Detailed Design

The proposed system presents a layered, behavior-based Endpoint Detection and Response (EDR) framework specifically designed for Industrial Control Systems and Operational Technology (ICS/OT) environments. The architecture emphasizes endpoint-level behavioral monitoring, lightweight anomaly detection, and operational safety, making it suitable for real-time industrial deployments. The overall design consists of five logical layers, as illustrated in the architecture diagram. Each layer performs a distinct function and interacts sequentially with adjacent layers to ensure effective monitoring and detection of anomalous behavior.

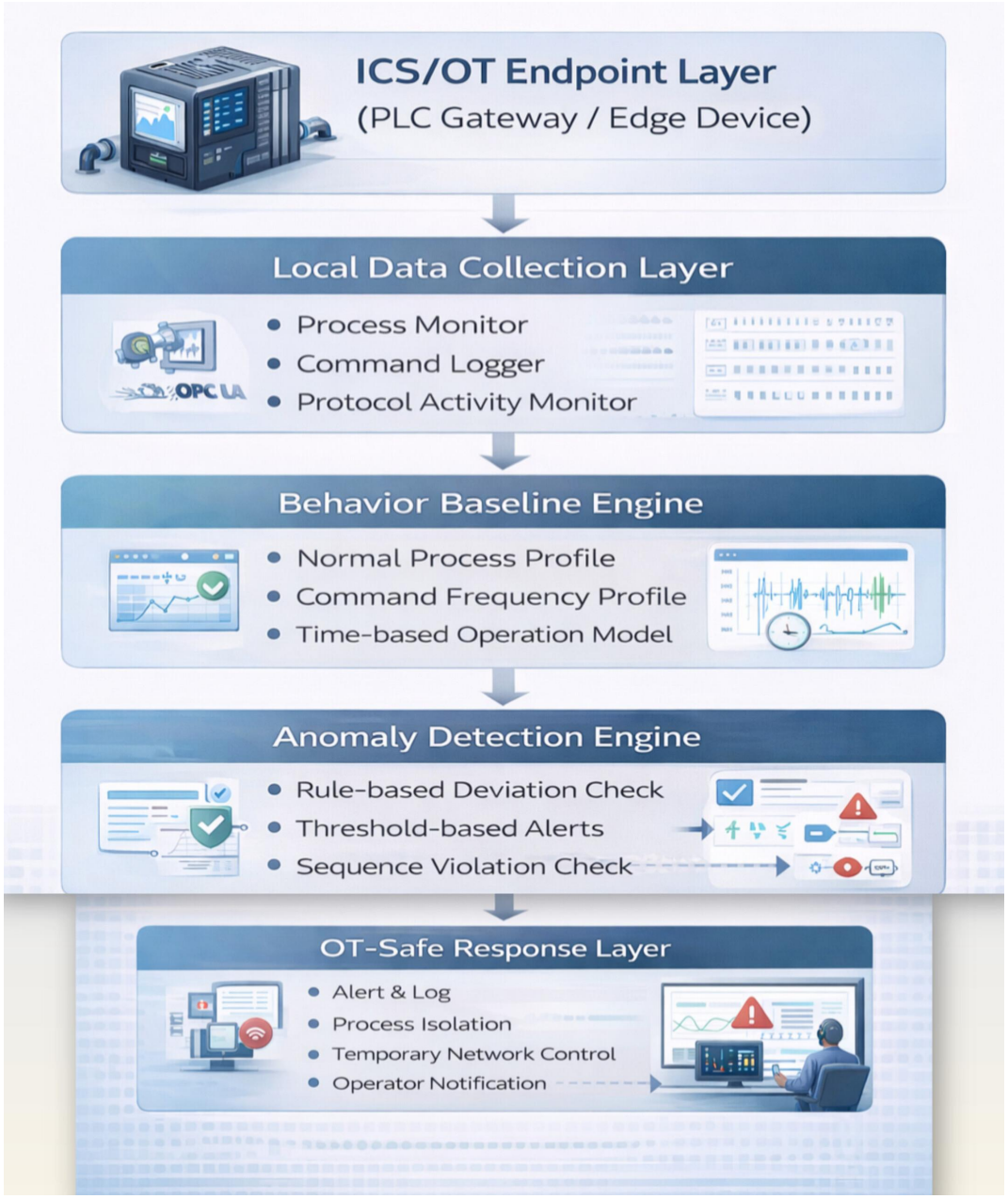

Figure 3.1: Architecture diagram

ICS/OT Endpoint Layer
The ICS/OT Endpoint Layer represents the industrial edge environment, such as a PLC gateway or edge device, responsible for executing control logic and interacting with sensors and actuators. This layer performs deterministic industrial operations and generates operational data during normal and abnormal conditions. In this work, the behavior of this layer is emulated using the Secure Water Treatment (SWaT) dataset, which reflects realistic industrial process behavior.

Local Data Collection Layer
The Local Data Collection Layer continuously monitors endpoint-level operational behavior without interfering with control processes. It includes three core components: a process monitor to observe execution patterns, a command logger to track actuator state changes, and a protocol activity monitor to capture communication timing and frequency. This layer focuses on metadata-level monitoring rather than payload inspection, ensuring minimal computational overhead and preserving system stability.

Behavior Baseline Engine
The Behavior Baseline Engine is responsible for modeling normal operational behavior of the ICS/OT endpoint. It constructs baseline profiles using historical normal-operation data, including normal process execution patterns, command frequency profiles, and time-based operation models. These baselines represent expected deterministic behavior and serve as reference points for detecting deviations during runtime.

Anomaly Detection Engine
The Anomaly Detection Engine analyzes real-time behavioral data against the established baselines to identify abnormal activities. Detection mechanisms include rule-based deviation checks, threshold-based alerts, and sequence violation analysis. This engine focuses on identifying behavioral anomalies such as abnormal command execution, unexpected timing variations, and deviations from normal operational sequences. Only a single anomaly detection engine is considered in the proposed design, as represented conceptually in the architecture.

OT-Safe Response Layer
The OT-Safe Response Layer is designed to ensure that detected anomalies are handled without disrupting industrial operations. Instead of aggressive mitigation actions, this layer provides controlled responses such as alert generation, event logging, logical process isolation, temporary network restrictions, and operator notification. This design ensures that security enforcement does not compromise safety or real-time performance requirements.
Overall, the proposed layered architecture enables effective detection of anomalous endpoint behavior while maintaining low overhead, high reliability, and operational safety. The modular design also allows for scalability and future enhancements without altering core industrial processes.

3.2 Design of Methodology

The proposed system follows a behavior-driven and layered methodology to detect anomalous activities in ICS/OT environments while maintaining operational safety. The methodology is designed to be lightweight, interpretable, and suitable for deterministic industrial processes.

Instead of relying on signature-based detection or complex machine learning models, the approach emphasizes behavioral baseline modeling and deviation analysis.

The methodology begins with the emulation of an ICS/OT endpoint using the Secure Water Treatment (SWaT) dataset, which provides realistic sensor and actuator behavior under normal and attack conditions. Endpoint-level operational data are first observed and collected through a local data collection mechanism. This data includes process execution characteristics, actuator state transitions, and time-based operational patterns.

In the next stage, the collected data is used to construct behavioral baselines that represent normal system operation. These baselines capture key characteristics such as normal process execution sequences, expected command frequencies, and timing constraints inherent to industrial control logic. The baseline models are generated exclusively from normal operational data to avoid bias introduced by attack scenarios.

Once the baseline profiles are established, incoming operational data are continuously compared against the baseline models using rule-based and threshold-based anomaly detection techniques. Deviations such as abnormal command frequency, unexpected sequence changes, or timing violations are identified as anomalous behavior. The detection results are then forwarded to the response layer, which is designed to support OT-safe mitigation actions. This methodology ensures effective anomaly detection with minimal computational overhead and without disrupting industrial processes.

3.3 Modules

The proposed system is organized into modular components to enhance clarity, scalability, and ease of implementation. Each module corresponds to a functional block in the system architecture and performs a specific role in the detection process.

ICS/OT Endpoint Module:
This module represents the industrial endpoint environment, such as a PLC gateway or edge device. It is responsible for executing control logic and interacting with sensors and actuators. In this project, the endpoint behavior is emulated using the SWaT dataset, which reflects realistic industrial operations.

Data Collection Module:
The data collection module continuously gathers endpoint-level operational data. It includes sub-components for monitoring process execution patterns, logging control command events through actuator state changes, and observing protocol-related timing behavior. This module operates in a passive manner to avoid interference with industrial operations.

Behavior Baseline Modeling Module:
This module constructs normal behavior profiles using historical operational data. It generates baseline models for process execution, command frequency, and time-based operation. These profiles define expected system behavior and serve as references for anomaly detection.

Anomaly Detection Module:
The anomaly detection module compares real-time operational behavior against established baseline profiles. It employs rule-based deviation checks, threshold-based alert mechanisms, and sequence validation to identify abnormal behavior indicative of potential cyber attacks or system misuse.

Response Interface Module:
Although not implemented in the current phase, this module defines the interface for OT-safe response actions. It is designed to generate alerts, log security events, and notify operators without enforcing disruptive control actions. The inclusion of this module ensures architectural completeness and supports future extensions.

3.4 Database Design

The database design of the proposed system focuses on storing and organizing endpoint-level behavioral data extracted during system operation. Since the project adopts a dataset-driven approach using the Secure Water Treatment (SWaT) dataset, the database is designed to efficiently manage structured logs related to process behavior, command execution, baseline profiles, and anomaly detection results.

The database follows a logical and lightweight design, suitable for behavior-based analysis rather than heavy transactional processing. It supports key operations such as data storage, retrieval, comparison, and analysis required during baseline modeling and anomaly detection. The stored data enables traceability of events and facilitates systematic evaluation of detected anomalies.The major data entities considered in the database include:

- Endpoint operational data
- Process behavior logs
- Control command logs
- Baseline behavior profiles
- Anomaly detection records

Each entity is designed to maintain temporal information, enabling time-based analysis and sequence validation, which are essential for ICS/OT behavior modeling. The database structure ensures minimal redundancy while maintaining clarity and scalability for future enhancements.

3.4.1 Entity Relationship Diagram

The Entity Relationship (ER) diagram represents the logical relationship between various data entities involved in the proposed system. The key entities and their relationships are described in the Figure 3.2 below.

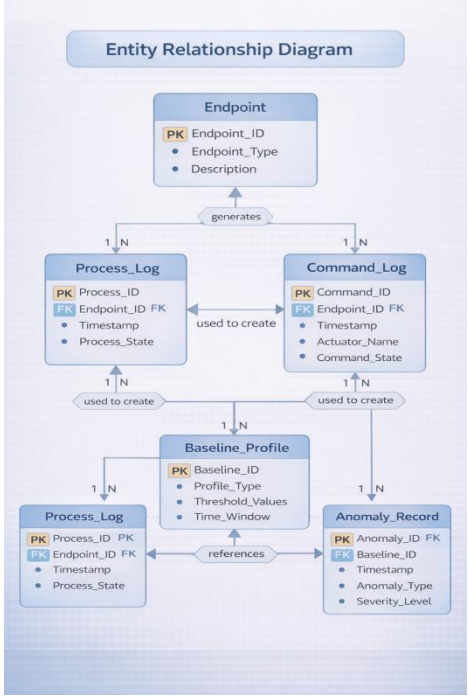

Figure 3.2: Entity diagram

Endpoint Entity

The Endpoint entity represents the ICS/OT endpoint, such as a PLC gateway or edge device. It acts as the source of all operational data.

- Attributes: Endpoint_ID, Endpoint_Type, Description

Process Log Entity

The Process Log entity stores information related to process execution patterns observed at the endpoint.

- Attributes: Process_ID, Endpoint_ID, Timestamp, Process_State
- Relationship: One Endpoint can generate many Process Logs (One-to-Many)

Command Log Entity

The Command Log entity captures control command events derived from actuator state changes.

- Attributes: Command_ID, Endpoint_ID, Timestamp, Actuator_Name, Command_State
- Relationship: One Endpoint can generate many Command Logs (One-to-Many)

Baseline Profile Entity

The Baseline Profile entity stores normal operational behavior models derived from historical data.

- Attributes: Baseline_ID, Profile_Type, Threshold_Values, Time_Window

- Relationship: One Baseline Profile is associated with multiple Process Logs and Command Logs (One-to-Many)

Anomaly Record Entity

The Anomaly Record entity stores information about detected deviations from baseline behavior.

- Attributes: Anomaly_ID, Baseline_ID, Timestamp, Anomaly_Type, Severity_Level
- Relationship: One Baseline Profile can be associated with multiple Anomaly Records (One-to-Many)

Chapter - 4
Implementation

4.1 Overview of Implementation

This chapter presents the detailed implementation of the proposed behavior-based Endpoint Detection and Response (EDR) framework for ICS/OT systems. The implementation follows a layered architecture, where each layer incrementally transforms raw industrial process data into actionable security intelligence. The system is implemented using a purely mathematical and rule-based approach, avoiding machine-learning models to ensure explainability, determinism, and OT safety. The SWaT dataset is used to emulate real ICS/OT endpoint behavior.

The implementation is organized into the following layers:

Layer 1: ICS/OT Endpoint Emulation

Layer 2: Local Data Collection

Layer 3: Behavior Baseline Engine

Layer 4: Anomaly Detection Engine

Layer 5: Response Engine

4.2 Layer 1 – ICS/OT Endpoint Emulation

4.2.1 Purpose

Layer 1 emulates the behavior of ICS/OT endpoints such as PLC-controlled actuators and sensors. Since deploying physical industrial hardware is complex and costly, the **SWaT dataset** is used as a realistic representation of endpoint behavior.

4.2.2 Implementation

1. The SWaT normal and attack datasets are loaded.
2. Each row represents a system snapshot at a given time.
3. Actuator and sensor variables represent endpoint-level behavior.

This layer does not apply any detection logic; it only provides raw operational data for further processing.

4.3 Layer 2 – Local Data Collection Layer

4.3.1 Purpose

The goal of Layer 2 is to reconstruct **endpoint-level behavioral logs** from raw process data. Instead of relying on network traffic or protocol parsing, this layer derives security-relevant metadata directly from control variable transitions.

4.3.2 Command Extraction Mechanism

Actuator commands in ICS systems are typically binary (e.g., ON/OFF, OPEN/CLOSE). Therefore, binary-valued variables are identified and treated as actuator control signals.

A command execution event is detected when an actuator changes its state.

4.3.3 Inter-Command Timing Calculation

To capture time-based behavior, the inter-command time is computed.

Mathematical Formulation

Let:

- t_i be the timestamp of the ith command.

$$\Delta t_i = t_i - t_{i-1}$$

Where:

- Δt_i represents the time interval between consecutive commands.

4.3.4 Command Frequency Measurement

The number of commands issued by each actuator is measured over fixed time windows.

Mathematical Formulation

Let:

- w denote a time window.

$$C_a(w) = \sum_{i \in w} 1$$

4.3.5 Output of Layer 2

Layer 2 generates structured logs containing:

- Actuator identity
- Timestamp of command execution
- Inter-command timing
- Command frequency per time window

4.3.5 Output of Layer 2

Layer 2 generates structured logs containing:

- Actuator identity
- Timestamp of command execution
- Inter-command timing
- Command frequency per time window

4.4.2 Timing Baseline Model

Mean Inter-Command Time

$$\mu_a = \frac{1}{N} \sum_{i=1}^{N} \Delta t_{a,i}$$

Standard Deviation

$$\sigma_a = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (\Delta t_{a,i} - \mu_a)^2}$$

4.4.3 Timing Bounds (3-Sigma Rule)

$$LB_a = \max(0, \mu_a - 3\sigma_a)$$

$$UB_a = \mu_a + 3\sigma_a$$

4.5 Layer 4 – Anomaly Detection Engine

4.5.1 Purpose

Layer 4 applies the baseline models to detect deviations in attack-phase data.

4.5.2 Timing-Based Detection

$$\Delta t_i < LB_a \quad \text{or} \quad \Delta t_i > UB_a$$

4.5.3 Frequency-Based Detection

$$C_a(w) > T_a$$

4.5.4 Identity-Based Detection

$$a \notin A_{\text{normal}}$$

4.5.5 Composite Anomaly Decision

$$\mathcal{A}(i) = \begin{cases} 1, & \text{if any anomaly is detected} \\ 0, & \text{otherwise} \end{cases}$$

This chapter demonstrated the complete implementation of a **mathematically driven EDR framework** for ICS/OT systems. The system:

- Reconstructs endpoint behavior from process data,
- Builds statistical baselines,
- Detects anomalies using deterministic rules,

Chapter - 5
Conclusion

This project presented a lightweight, behavior-based Endpoint Detection and Response (EDR) framework designed specifically for Industrial Control Systems and Operational Technology (ICS/OT) environments. The proposed system addresses key limitations of traditional IT-centric security solutions by focusing on deterministic operational behavior and endpoint-level monitoring rather than signature-based detection or computationally intensive learning techniques.

A layered architecture was designed and implemented up to the anomaly detection stage, comprising an ICS/OT endpoint layer, local data collection, behavior baseline modeling, and anomaly detection. The Secure Water Treatment (SWaT) dataset was used to emulate real-world industrial operations and evaluate the effectiveness of the proposed approach. Behavioral baselines were constructed using normal operational data, and deviations were identified using rule-based, frequency-based, timing-based, and sequence-based detection mechanisms.

The results demonstrate that the proposed framework can effectively detect anomalous behaviors with minimal computational overhead while preserving operational safety. By avoiding aggressive response actions and complex models, the system remains suitable for real-time and safety-critical industrial environments.

In summary, this work highlights the effectiveness of behavior-based endpoint monitoring for ICS/OT security and provides a strong foundation for future enhancements, including OT-safe response automation, adaptive thresholding, and integration with real-time industrial monitoring platforms.

References

Amer, E., Al-Rimy, B.A.S. and El-Sappagh, S. (2025) 'Strengthening ICS defense: Modbus-NFA behavior model for enhanced anomaly detection', *Journal of Network and Computer Applications*, 237, pp. 1–18. Available at: https://www.sciencedirect.com/science/article/pii/S2214212625000286 (Accessed: 2026).

Chen, T., Zheng, C., Zhu, T., Xiong, C., Ying, J., Yuan, Q., Cheng, W. and Lv, M. (2023) 'System-level data management for endpoint advanced persistent threat detection: Issues, challenges and trends', *Computers & Security*, 129, pp. 1–22. Available at: https://www.sciencedirect.com/science/article/abs/pii/S0167404823003954 (Accessed: 2026).

Imran, M., Siddiqui, H.U.R., Raza, A., Raza, M.A., Rustam, F. and Ashraf, I. (2023) 'A performance overview of machine learning-based defense strategies for advanced persistent threats in industrial control systems', *Computers & Security*, 126, pp. 1–17. Available at: https://www.sciencedirect.com/science/article/abs/pii/S0167404823003553 (Accessed: 2026).

Kuchar, K. and Fujdiak, R. (2025) 'Analyzing anomalies in industrial networks: A data-driven approach to enhance security in manufacturing processes', *Computers & Security*, 137, pp. 1–20. Available at: https://www.sciencedirect.com/science/article/pii/S0167404825000847 (Accessed: 2026).

Qin, X., Doss, R., Jiang, F., Qin, X. and Long, B. (2025) 'Securing ICS networks: SDN-based automated traffic control and moving target defense framework against DDoS attacks', *Computer Communications*, 217, pp. 1–15. Available at: https://www.sciencedirect.com/science/article/pii/S0140366425002099 (Accessed: 2026).

Zare, F., Mahmoudi-Nasr, P. and Yousefpour, R. (2024) 'A real-time network-based anomaly detection in industrial control systems', *Journal of Industrial Information Integration*, 38, pp. 1–12. Available at: https://www.sciencedirect.com/science/article/abs/pii/S1874548224000179 (Accessed: 2026).

Zhu, Q., Ding, Y., Jiang, J. and Yang, S.-H. (2025) 'Anomaly detection using invariant rules in industrial control systems', *Control Engineering Practice*, 146, pp. 1–14. Available at: https://www.sciencedirect.com/science/article/pii/S096706612400323X (Accessed: 2026).

Appendix A - Source code

```
# %%
import pandas as pd

# Load SWaT datasets
normal_df = pd.read_csv("normal.csv")
attack_df = pd.read_csv("attack.csv")
```

```
# Basic inspection
print("Normal dataset shape:", normal_df.shape)
print("Attack dataset shape:", attack_df.shape)
```

```
normal_df.head()
```

```
# %%
normal_df.columns
```

```
# %%
# Identify time column
time_col = normal_df.columns[0]
print("Time column:", time_col)
```

```
# Count actuator-like variables (binary)
binary_cols = normal_df.columns[(normal_df.nunique() <= 2)]
print("Binary (actuator-like) variables:", len(binary_cols))
```

```
# %%
# Select only numeric columns
numeric_df = normal_df.select_dtypes(include=["number"])
```

```
# From numeric columns, select binary (0/1) columns
binary_cols = [
    col for col in numeric_df.columns
    if numeric_df[col].nunique() <= 2
]
```

```
print("Number of binary actuator columns:", len(binary_cols))
binary_cols[:10]
```

```
# %%
command_events = []
```

```
for col in binary_cols:
    changes = numeric_df[col].diff().fillna(0) != 0
    events = normal_df.loc[changes, [time_col]]
    events["actuator"] = col
    events["new_state"] = normal_df.loc[changes, col]
    command_events.append(events)
```

```
command_log = pd.concat(command_events, ignore_index=True)
command_log.head()
```

```
# %%
import os
```

```
# Create layer2_logs directory if it doesn't exist
os.makedirs("../layer2_logs", exist_ok=True)
```

```
# %%
command_log.to_csv("../layer2_logs/command_logs_normal.csv", index=False)
```

```
# %%
os.listdir("../layer2_logs")
```

```
# %%
command_log["actuator"].value_counts().head()
```

```
# %%
# Convert timestamp to datetime
command_log[time_col] = pd.to_datetime(command_log[time_col])
```

```
command_log.head()
```

```
# %%
# Convert timestamp to datetime
command_log[time_col] = pd.to_datetime(command_log[time_col])
```

```
command_log.head()
```

```
# %%
command_log.columns
```

```
# %%
# Make sure timestamp is datetime
command_log[time_col] = pd.to_datetime(command_log[time_col])
```

```
# Sort by time (VERY IMPORTANT)
command_log = command_log.sort_values(by=time_col)
```

```
# Create delta_t column
command_log["delta_t"] = command_log[time_col].diff().dt.total_seconds()
```

```
command_log[["actuator", "delta_t"]].head()
```

```
# %%
timing_profile = (
    command_log
    .groupby("actuator")["delta_t"]
    .agg(["mean", "std", "min", "max"])
    .reset_index()
)
```

```
timing_profile.head()
```

```
# %%
timing_profile.sort_values("mean").head()
```

```
# %%
# Set timestamp as index
command_log = command_log.set_index(time_col)
```

```
# Commands per minute per actuator
frequency_log = (
    command_log
    .groupby("actuator")
    .resample("1min")
    .size()
    .reset_index(name="command_count")
)
```

```
frequency_log.head()
```

```
# %%
timing_profile.to_csv("../layer2_logs/timing_profile.csv", index=False)
frequency_log.to_csv("../layer2_logs/frequency_log.csv", index=False)
```

```
# %%
import matplotlib.pyplot as plt
```

```
# Top 10 most active actuators
top_actuators = (
    command_log["actuator"]
    .value_counts()
    .head(10)
)
```

```
plt.figure()
top_actuators.plot(kind="bar")
plt.xlabel("Actuator")
plt.ylabel("Number of Commands")
plt.title("Top Actuators by Command Frequency")
plt.show()
```

```
# %%
plt.figure()
command_log["delta_t"].dropna().hist(bins=50)
plt.xlabel("Inter-command Time (seconds)")
plt.ylabel("Frequency")
plt.title("Distribution of Inter-command Timing (Δt)")
plt.show()
```

```
# %%
# Reset index if needed
command_log = command_log.reset_index()
```

```
plt.figure()
plt.plot(command_log[time_col], range(len(command_log)))
plt.xlabel("Time")
plt.ylabel("Cumulative Command Count")
plt.title("Command Execution Timeline")
```

```
plt.show()
```

```
# %%
plt.savefig("Images/command_frequency.png", dpi=300, bbox_inches="tight")
```

```
# %%
timing_profile.columns
```

```
# %%
# Create timing bounds again
timing_profile["lower_bound"] = timing_profile["mean"] - 3 * timing_profile["std"]
timing_profile["upper_bound"] = timing_profile["mean"] + 3 * timing_profile["std"]
```

```
# Clamp negative lower bounds (OT-safe)
timing_profile["lower_bound"] = timing_profile["lower_bound"].clip(lower=0)
```

```
# %%
timing_profile[["actuator", "mean", "std", "lower_bound", "upper_bound"]].head()
```

```
# %%
timing_profile.describe()
```

```
# %%
import matplotlib.pyplot as plt
```

```
# Pick one frequent actuator
act = timing_profile.sort_values("mean").iloc[0]["actuator"]
```

```
# Extract data
act_data = command_log[command_log["actuator"] == act]["delta_t"].dropna()
lb = timing_profile.loc[timing_profile["actuator"] == act, "lower_bound"].values[0]
ub = timing_profile.loc[timing_profile["actuator"] == act, "upper_bound"].values[0]
```

```
plt.figure()
plt.hist(act_data, bins=50)
plt.axvline(lb)
plt.axvline(ub)
plt.xlabel("Inter-command Time (seconds)")
plt.ylabel("Frequency")
plt.title(f"Timing Baseline Validation for {act}")
plt.show()
```

```
# %%
locals().keys()
```

```
# %%
# Recreate frequency baseline from frequency_log
freq_baseline = (
    frequency_log
    .groupby("actuator")["command_count"]
    .agg(["mean", "std", "max"])
    .reset_index()
)
```

```
# Define frequency threshold
freq_baseline["freq_threshold"] = freq_baseline["mean"] + 3 * freq_baseline["std"]
```

```
freq_baseline.head()
```

```
# %%
freq_check = frequency_log.merge(
    freq_baseline[["actuator", "freq_threshold"]],
    on="actuator",
    how="left"
)
```

```
freq_violations = freq_check[
    freq_check["command_count"] > freq_check["freq_threshold"]
]
```

```
freq_violations.head()
```

```
# %%
# Recreate allowed actuator set from NORMAL command log
allowed_actuators = set(command_log["actuator"].unique())
```

```
allowed_actuators
```

```
# %%
unexpected = set(command_log["actuator"]) - allowed_actuators
unexpected
```

```
# %% [markdown]
# Response Level 1 – Alert & Logging
```

```
# %%
def response_level_1(event):
    return {
        "response_level": 1,
        "action": "Alert generated",
        "recommendation": "Monitor actuator behavior"
    }
```

```
# %% [markdown]
# Response Level 2 – Controlled Restriction
```

```
# %%
def response_level_2(event):
    return {
        "response_level": 2,
        "action": "Suspicious activity flagged",
        "recommendation": "Investigate actuator or control logic"
    }
```

```
# %% [markdown]
# Response Level 3 – Critical Alert
```

```
# %%
```

```
def response_level_3(event):
    return {
        "response_level": 3,
        "action": "Critical anomaly detected",
        "recommendation": "Immediate operator intervention required"
    }
```

```
# %%
# ========== REBUILD ALL LAYER-4 ANOMALIES SAFELY ==========
```

```
# Ensure timestamp is datetime
attack_df[time_col] = pd.to_datetime(attack_df[time_col])
```

```
# ---------- Timing anomalies ----------
timing_check = attack_command_log.merge(
    timing_baseline[["actuator", "lower_bound", "upper_bound"]],
    on="actuator",
    how="left"
)
```

```
timing_anomalies = timing_check[
    (timing_check["delta_t"] < timing_check["lower_bound"]) |
    (timing_check["delta_t"] > timing_check["upper_bound"])
]
```

```
# ---------- Frequency anomalies ----------
freq_check = attack_freq.merge(
    freq_baseline[["actuator", "freq_threshold"]],
    on="actuator",
    how="left"
)
```

```
freq_anomalies = freq_check[
    freq_check["command_count"] > freq_check["freq_threshold"]
]
```

```
# ---------- Identity anomalies ----------
identity_anomalies = attack_command_log[
    ~attack_command_log["actuator"].isin(allowed_actuators)
]
```

```
# ---------- Collect all anomaly timestamps ----------
anomaly_times = set()
```

```
anomaly_times.update(timing_anomalies[time_col])
anomaly_times.update(freq_anomalies[time_col])
anomaly_times.update(identity_anomalies[time_col])
```

```
len(anomaly_times)
```